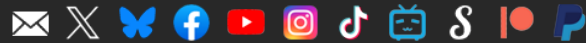


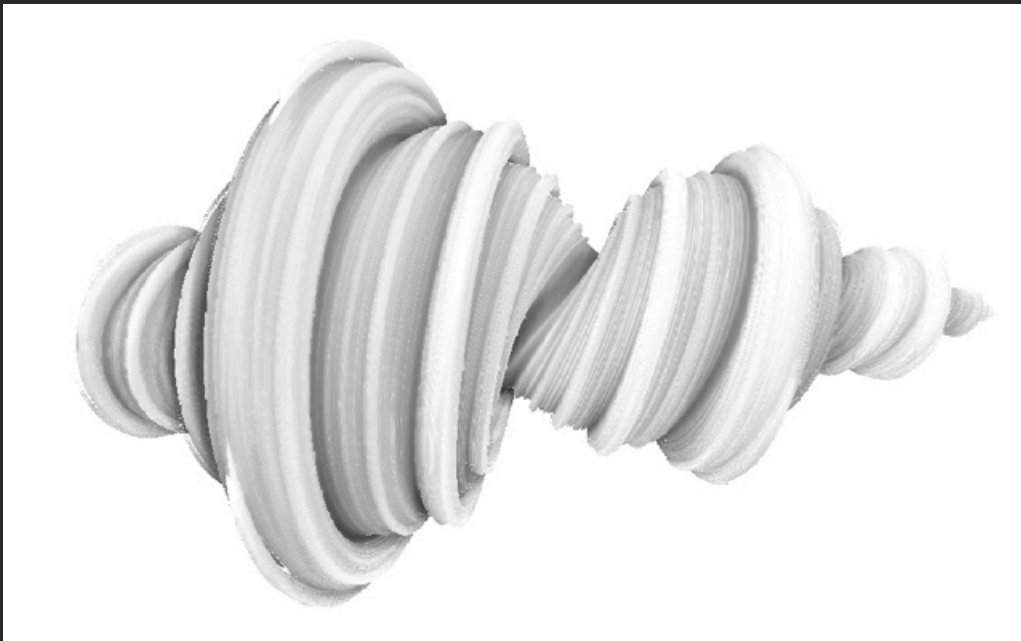
Inigo Quilez :: articles :: screen space ambient occlusion - 2007



Disclaimer - I wrote this article almost 20 years ago, and it was probably the first public documentation exploring and describing the technique. Naturally the technique has evolved since then (although its main problem still remains unresolved), so read this article in its historical context.

SSAO stands for **S**creen **S**pace **A**mbient **O**clusion, and it partially makes reality one of the deepest dreams of computer graphics programmers: ambient occlusion in realtime, mine included (see [here](#)). The term was first used by Crytek when they introduced it in a small paragraph of a paper called "Finding Next Gen: CryEngine 2" (just google for that sentence).

Since then many CG programming enthusiast tried to decipher how the technique works, and each one got different results with varying quality and performances. I did my own investigations and I arrived to a method that being not optimal, still gives cool results and is quite usable. Of course I went thru many revisions of the algorithm, but well, this is the technique I used for **Kindernoiser** and **Kindercrasher** 4 kilobyte demos.



Screen Space Ambient Occlusion term applied to a complex shape

The trick:

Ambient occlusion, as other direct lighting techniques (and indirect too of course) is based on a non-local computations. This means it's not enough to know the surface properties of the point to be shaded, but one needs some description of the surrounding geometry as well. Since this information is not accessible on modern rasterization hardware (that's why we will never see good realtime shadows in OpenGL or DirectX), the Crytek team (as many other guy somehow before them) came with the idea to use the zbuffer to partially recover such information. Zbuffer can be seen as a small repository of geometry information: from each pixel on the buffer one can recover the 3d position of the geometry (well, the closest to the camera surface) projected on that pixel.

Thus the idea is to use that information in a two (or more) pass algorithm. First render the scene normally, or almost, and in a second full screen quad pass compute the ambient occlusion at each pixel and use it to modify the already computed lighting. For that, for each pixel for which we compute the AO we construct few 3d points around it and see if these points are occluded from camera's point of view. This is not ambient occlusion as in the usual definition, but it indeed gives some kind of concavity for the shaded point, what can be interpreted as an (ambient) occlusion factor.

To simplify computations on the second pass, the first pass outputs a linear eye space z distance (instead of the $1/z$ used on the regular zbuffers). This is done per vertex since z, being linear, can be safely interpolated on the surface of the polygons. By using multiple render targets one can output this buffer at the same time as the regular color buffer.

The second pass draws a screen space polygon covering the complete viewport and performs the ambient occlusion computation. For that it first recovers the eye space position of each pixel by unprojection: it reads the z value from the previously prepared texture, and given the eye space view vector (computed by interpolation from the vertex shader) it computes the eye space position. Say **gl_TexCoord[0]** contains the eye view vector (not necessarily normalized), **tex0** the linear zbuffer, and **gl_Color** the 2d pixel coordinates (from 0 to 1 for the complete viewport), then:

```
float ez = texture2D( tex0, gl_Color.xy );           // eye z distance
vec3 ep = ez*gl_TexCoord[0].xyz/gl_TexCoord[0].z;    // eye point
```

next we have to generate N 3d points. I believe Crytek uses 8 points for low end machines (pixel shaders 2.0)

and 16 for more powerful machines. It's a trade off between speed and quality, so definitively a parameter to play with. I generated the points around the current shading point in a sphere (inside the sphere, not just on the surface) from a small random lookup table (passed as constants), with a constant radius (scene dependent, and feature dependent - you can make the AO more local or global by adjusting this parameter).

```
for( int i=0; i<32; i++ )
{
    vec3 se = ep + rad*fk3f[i].xyz;
```

Next we project these points back into clip space with the usual perspective division and look up on the zbuffer for the scene's eye z distance at that pixel, as in shadow mapping:

```
vec2 ss = (se.xy/se.z)*vec2(0.75,1.0);
vec2 sn = ss*0.5 + vec2(0.5);
vec4 sz = texture2D(tex0,sn);
```

or alternatively

```
vec3 ss = se.xyz*vec3(0.75,1.0,1.0);
vec4 sz = texture2DProj( tex0, ss*0.5+ss.z*vec3(0.5) );
```

Now the most tricky part of the algorithm comes. Unlike in shadow mapping where a simple comparison yields a binary value, in this case we have to be more careful because we have to account for occlusion, and occlusion factor are distance dependent while shadows are not. For example, a surface element that is far from the point under consideration will occlude less that point than if it was closer, with a quadratic attenuation (have a look [here](#)). So this means it should be a bit like a step() curve so that for negative values it does not occlude, but it should then slowly attenuate back to zero also. The attenuation factor, again, depends on the scale of the scene and aesthetical factors. I call this function "blocking or occlusion function". The idea is then to accumulate the occlusion factor, like:

```
float zd = 50.0*max( se.z-sz.x, 0.0 );
b1 += 1.0/(1.0+zd*zd);
```

and to finish we just have to average to get the total estimated occlusion.

```
}
gl_FragColor = vec4(b1/32.0);
```

The second trick:

Doing it as just described creates some ugly banding artifacts derived from the low sampling rate of the occlusion (32 in the example above, 8 or 16 in Crytek's implementation). So the next step is to apply some dithering to the sampling pattern. Crytek suggests to use a per pixel random plane to do a reflection on the sampling point around the shading point, what works very well in practice and is very fast. For that we have to prepare a small random normal map, accessible thru **tex1** on the following modified code:

```
vec3 se = ep + rad*reflect(fk3f[i].xyz,pl.xyz);
```

so the complete shader looks like:

```
uniform vec4 fk3f[32];
uniform vec4 fres;
uniform sampler2D tex0;
uniform sampler2D tex1;

void main( void )
{
    vec4 zbu = texture2D( tex0, gl_Color.xy );

    vec3 ep = zbu.x*gl_TexCoord[0].xyz/gl_TexCoord[0].z;

    vec4 pl = texture2D( tex1, gl_Color.xy*fres.xy );
    pl = pl*2.0 - 1.0;

    float b1 = 0.0;
    for( int i=0; i<32; i++ )
    {
        vec3 se = ep + rad*reflect(fk3f[i].xyz,pl.xyz);

        vec2 ss = (se.xy/se.z)*vec2(0.75,1.0);
        vec2 sn = ss*0.5 + vec2(0.5);
        vec4 sz = texture2D(tex0,sn);

        float zd = 50.0*max( se.z-sz.x, 0.0 );
        b1 += 1.0/(1.0+zd*zd);
    }
    gl_FragColor = vec4(b1/32.0);
}
```

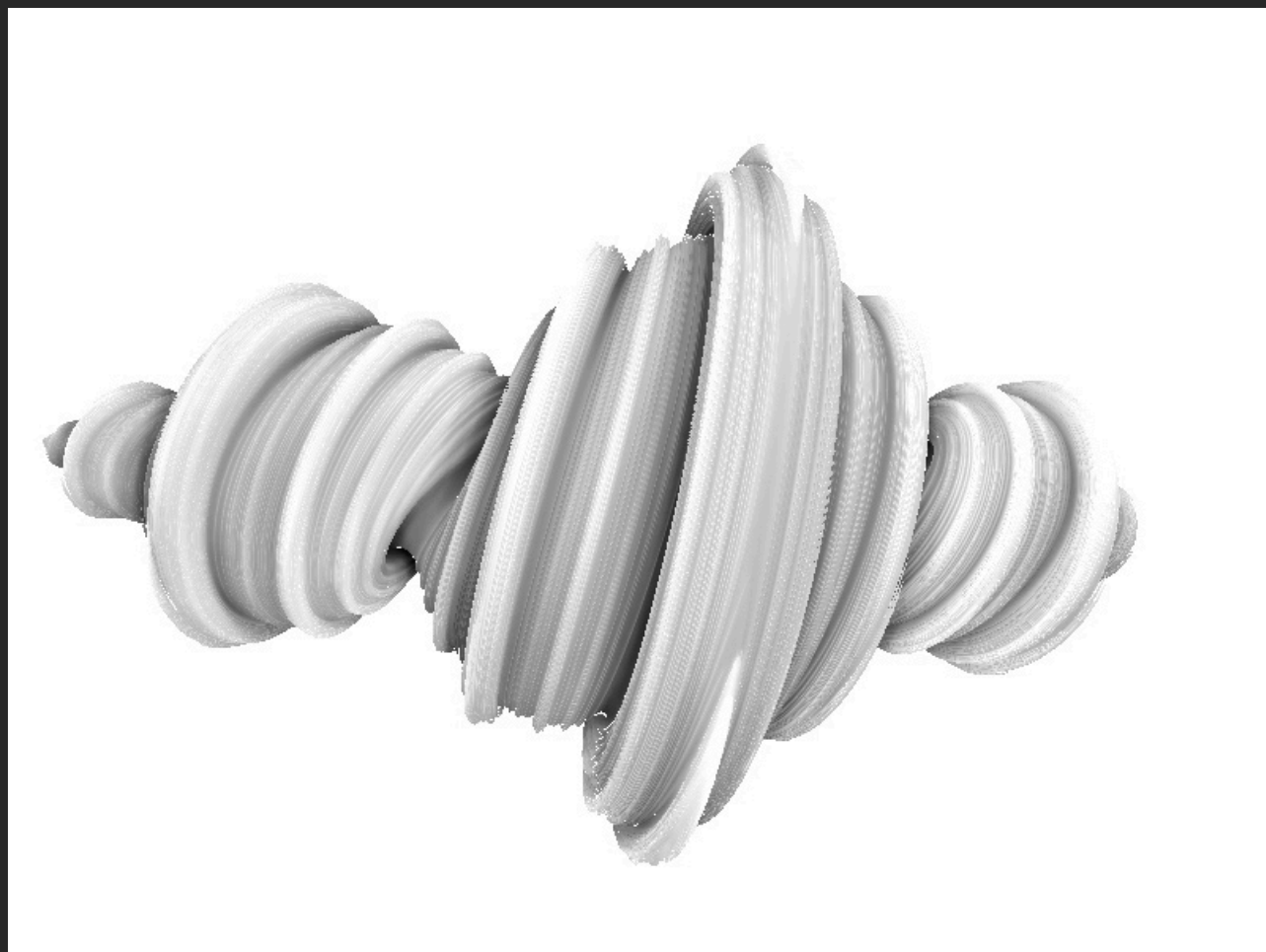
The big secret trick is to apply next a blurring to the ambient occlusion, that we stored in a texture (occlusion map). It's easy to avoid blurring across object edges by checking the difference in z between the blurring sampling points, and the eye space normal too (that we can output as with the eye linear z distance on the very first pass).

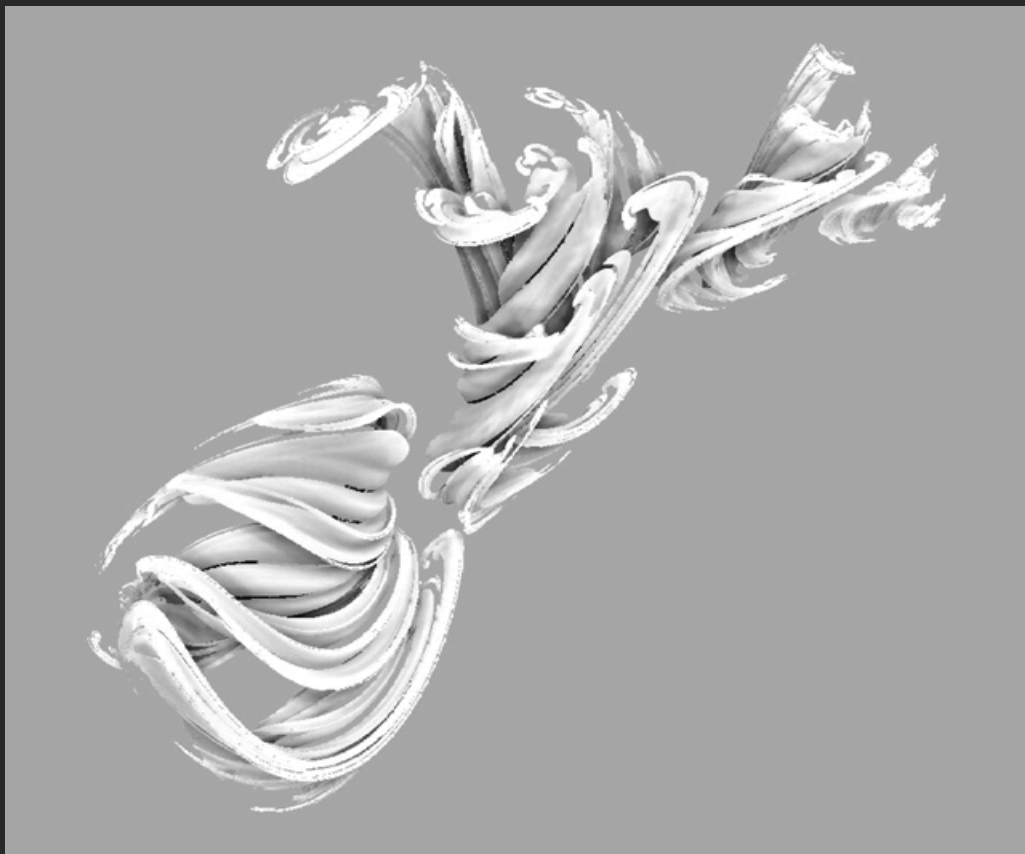
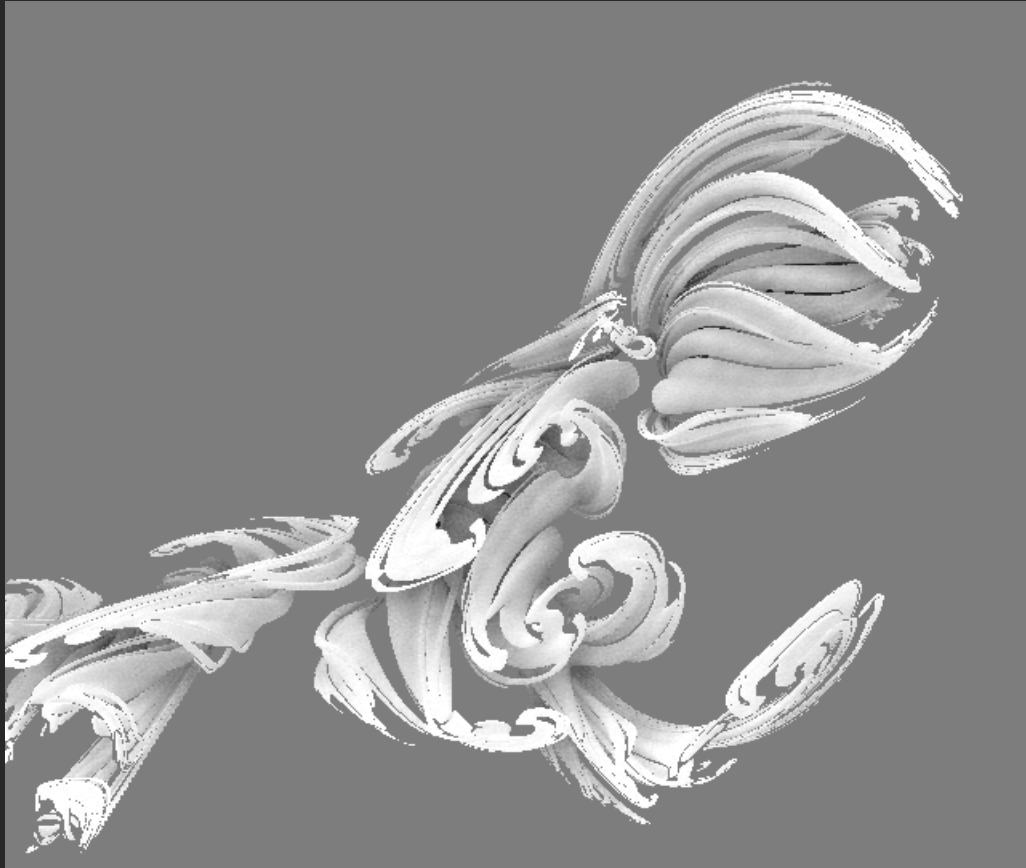
Optimizations:

The shader above does not execute in pixel shaders 2.0 hardware, because of the amount of instructions, even with just 8 sampling points, while Crytek's does. So, the thing is to simplify the inner loop code. The first this one can do is to remove the perspective projection applied to the sampling points. This has a nice side effect, and it's that the sampling sphere is constant size in screen space regardless the distance to the camera, what allows for ambient occlusion both in close and distant objects at the same time. That's what Crytek guys do, I believe. Once could play with the blocking factor to remove few instructions too.

Results:

I added few reference images below of this realtime **Screen Space Ambient Occlusion** implementation above:





inigo quilez - learning computer graphics since 1994